

[Menu](#)

Data Management

▼ Relevant source files

`admin/setup/dboperations/dailyCronJobs.php``admin/utilities/cleanupFoldersDBs.php``admin/utilities/notifyDatabaseArchive.php``admin/utilities/purgeInactiveDBs.php``admin/verification/checkRecURL.php``export/dbbackup/buildArchivePackagesCMD.php``export/dbbackup/exportMyDataPopup.php``export/xml/flathml.php``hclient/widgets/database/dbAction.js``hclient/widgets/database/dbVerify.html``hserv/controller/databaseController.php``hserv/structure/export/getDBStructureAsSQL.php``hserv/structure/export/getDBStructureAsXML.php``import/delimited/importRecordsCSV.js``import/delimited/importRecordsCSV.php``import/utilities/importRecords.js``import/utilities/importRecords.php`

This page provides an overview of Heurist's data management capabilities, including importing data from external sources, exporting data for backup or exchange, and database management operations. These features allow users to get data into and out of the system, maintain database integrity, and perform administrative tasks on databases.

For information about database structure and configuration, see [Database Structure](#).

Import System

Heurist provides robust facilities for importing data from various formats into the database. The import system allows data to be mapped to existing record types and fields, with options for updating existing records or creating new ones.

CSV/TSV Import

The CSV/TSV import facility allows importing tabular data from spreadsheets or other data sources. It supports various encoding formats, delimiters, and field mappings.

! CSV/TSV Import Process flowchart TD subgraph "CSV/TSV Import Process" uploadFile["File Upload"] → parseFile["Parse File"] parseFile → analyzeHeader["Analyze Header"] analyzeHeader → identifyColumns["Identify Columns"] identifyColumns → selectRecType["Select Record Type"] selectRecType → mapFields["Map Fields"] mapFields → match["Match Records"] match → validate["Validate Data"] validate → import["Import Data"] end

The import process involves:

1. **Upload**: Users upload a CSV/TSV file or paste data directly
2. **Encoding Selection**: Choose file encoding, field separators, multi-value separators
3. **Field Identification**: Identify which columns contain record IDs, dates, etc.
4. **Primary Record Type Selection**: Determine the main record type for import
5. **Field Mapping**: Map CSV columns to Heurist fields
6. **Matching**: Match imported data against existing records (optional)
7. **Data Validation**: Validate data format and integrity
8. **Import**: Create new records or update existing ones

The system supports:

- Multiple encodings (UTF-8, ASCII, etc.)
- Various field separators (comma, tab, semicolon, etc.)
- Different date formats
- Multi-value fields with custom separators
- ID fields for record matching

Sources: <FileRef file-url="https://github.com/HeuristNetwork/heurist/blob/c9f37be9/import/delimited/importRecon"

XML/JSON Import

Heurist supports importing data from XML (HML format) or JSON files, typically exported from other Heurist datab

!<FileRef file-url="https://github.com/HeuristNetwork/heurist/blob/c9f37be9/XML/JSON Import Process" undefined flowchart TD

```
subgraph "XML/JSON Import Process"
```

```
  upload["File Upload"] --> analyze["Analyze Structure"]
```

```
  analyze --> syncDefs["Sync Definitions"]
```

```
  syncDefs --> importRecs["Import Records"]
```

```
  analyze -- "Missing definitions" --> downloadDefs["Download Definitions"]
```

```

downloadDefs --> syncDefs

subgraph "Record Import Options"
    insertOnly["Insert Only"]
    updateOnly["Update Only"]
    insertUpdate["Insert & Update"]
end

importRecs --> insertOnly
importRecs --> updateOnly
importRecs --> insertUpdate
end

```

The XML/JSON import process includes:

1. **File Upload:** Users upload an XML or JSON file
2. **Structure Analysis:** The system analyzes source database structure
3. **Definition Synchronization:** Missing record types, fields, or terms are identified and can be imported
4. **Record Import:** Data is imported according to user-selected options

The system provides options for:

- Synchronizing database definitions (record types, fields, terms)
- Matching records using unique ID fields
- Inserting new records and/or updating existing ones
- Handling data from non-registered databases

Sources: [import/utilities/importRecords.php](#) 1-108 [import/utilities/importRecords.js](#) 1-166

Export System

Heurist offers several methods for exporting data, enabling backup, data exchange, and interoperability with other systems.

XML/HML Export

The HML (Heurist Markup Language) export produces an XML representation of records with options for including related records and file content.

! **HML Export Process** flowchart LR subgraph "HML Export Process" query["Search Query"] → filter["Apply Filters"] filter → generateXML["Generate XML"] generateXML → outputFile["Output File"]

```

subgraph "Export Options"
    depth["Traversal Depth"]
    reverse["Include Reverse Pointers"]
    expand["Expand Related Records"]
    fileContent["Include File Content"]
end

depth --> filter
reverse --> filter
expand --> generateXML
fileContent --> generateXML
end

```

HML export features include:

1. **Record Selection**: Export specific records based on search criteria
2. **Relationship Traversal**: Include related records to a specified depth
3. **Pointer Handling**: Include reverse pointers and relationships
4. **File Content**: Option to include file/media content
5. **Format Control**: Options for output formatting and structure

Parameters for controlling export:

- `depth`: How many levels of related records to include (0-n or "all")
- `rev`: Include reverse pointers (yes/no)
- `revexpand`: Expand reverse pointers (yes/no)
- `linkmode`: How to handle links (none, direct, direct_links, all)
- `fc`: Include file content (0/1)

Sources: <FileRef file-url="https://github.com/HeuristNetwork/heurist/blob/c9f37be9/export/xml/flathml.php#L1-L2

Database Backup and Archiving

Heurist provides comprehensive database backup and archiving capabilities, allowing creation of self-contained p

!<FileRef file-url="https://github.com/HeuristNetwork/heurist/blob/c9f37be9/Database Backup Process" undefined
flowchart TD

```

subgraph "Database Backup Process"
    initBackup["Initialize Backup"] --> copySystem["Copy System Files"]
    copySystem --> copyUploaded["Copy Uploaded Files"]
    copyUploaded --> createSQL["Create SQL Dump"]
    createSQL --> generateHML["Generate HML"]
    generateHML --> exportTSV["Export TSV"]
    exportTSV --> packageZip["Package ZIP"]
end

```

The backup system generates comprehensive packages that include:

1. **SQL Database Dump**: Complete database structure and data in SQL format
2. **HML Export**: Self-documenting XML representation of all records (optional)

3. **TSV Export:** Tabular representation of records (optional)
4. **Uploaded Files:** All files uploaded to the database (optional)
5. **Documentation:** Background documentation about Heurist structure and formats
6. **Archive Package:** All components packaged in a ZIP or BZip format

The archiving system supports:

- Excluding large files (e.g., tilestacks) to reduce package size
- Selective inclusion of components (SQL, HML, TSV, files)
- Archive creation via user interface or command-line
- Repository upload capabilities

Sources: `export/dbbackup/exportMyDataPopup.php` 1-134

`export/dbbackup/exportMyDataPopup.php` 300-380

`export/dbbackup/buildArchivePackagesCMD.php` 1-93

Database Management

Heurist includes a suite of tools for database management, including creation, cloning, verification, and maintenance operations.

Database Operations

The system provides operations for creating, cloning, renaming, and deleting databases.

! Database Operations flowchart TD subgraph "Database Operations" create["Create Database"] clone["Clone Database"] rename["Rename Database"] clear["Clear Database"] delete["Delete Database"] restore["Restore from Archive"]

```
create --> newDB["New Database"]
clone --> newDB
rename --> newDB
restore --> newDB

clear --> existingDB["Existing Database"]
delete -- "Creates backup" --> archiveDB["Archived Database"]
end
```

Key database operations include:

1. ****Create**:** Create a new empty database with core definitions
2. ****Clone**:** Create a copy of an existing database (with or without data)

3. ****Rename****: Change the name of a database
4. ****Clear****: Remove all records while keeping the structure
5. ****Delete****: Delete a database (with option to create an archive first)
6. ****Restore****: Restore a database from an archive file

Each operation includes appropriate security checks and user permission verification. Database names must follow

Sources: <FileRef file-url="https://github.com/HeuristNetwork/heurist/blob/c9f37be9/hserv/controller/databaseCon

Database Verification

Heurist provides tools to verify database integrity, identify issues, and validate external resources such as UR

!<FileRef file-url="https://github.com/HeuristNetwork/heurist/blob/c9f37be9/Database Verification" undefined fi
flowchart TD

```

subgraph "Database Verification"
    verifyIntegrity["Verify Database Integrity"]
    checkURLs["Check URLs"]
    verifyFiles["Verify Files"]

    verifyIntegrity --> checkRecords["Check Records"]
    verifyIntegrity --> checkFields["Check Fields"]
    verifyIntegrity --> checkTerms["Check Terms"]
    verifyIntegrity --> checkPointers["Check Pointers"]

    checkURLs --> validateRecURLs["Validate Record URLs"]
    checkURLs --> validateFieldURLs["Validate URLs in Fields"]
    checkURLs --> validateFileURLs["Validate File URLs"]

    verifyFiles --> findMissing["Find Missing Files"]
    verifyFiles --> findDuplicate["Find Duplicate Files"]
    verifyFiles --> findUnused["Find Unused Files"]
end

```

end

Verification capabilities include:

1. Database Integrity Checks:

- Validation of record structure
- Verification of field values and required fields
- Checking of pointer references
- Term validation

2. URL Validation:

- Checking record URLs (rec_URL field)
- Validating URLs in text fields
- Verifying file URLs

3. File System Verification:

- Finding missing uploaded files
- Identifying duplicate files
- Detecting unused files

URL validation can be run manually or as part of scheduled maintenance tasks. The system reports invalid URLs for administrator review.

Sources: `admin/verification/checkRecURL.php` 1-84 `hclient/widgets/database/dbVerify.html` 1-58

Database Maintenance

Heurist includes maintenance utilities for database cleanup, purging inactive databases, and managing system resources.

! Database Maintenance flowchart TD subgraph "Database Maintenance" cleanupFolders["Cleanup Folders"] purgeInactive["Purge Inactive DBs"] archiveImports["Archive Import Tables"] purgeSysArchive["Purge System Archive"]

```
cleanupFolders --> removeScratch["Remove Scratch Files"]
cleanupFolders --> removeBackups["Remove Old Backups"]
cleanupFolders --> removeTemp["Remove Temp Files"]

purgeInactive --> archiveSmall["Archive Small Inactive DBs"]
purgeInactive --> notifyLarge["Notify About Large Inactive DBs"]

archiveImports --> compressOld["Compress Old Import Tables"]
archiveImports --> limitTables["Limit Number of Tables"]

purgeSysArchive --> compressArchive["Compress System Archive"]
end
```

Maintenance operations include:

- Folder Cleanup:**
 - Removal of temporary files
 - Cleanup of scratch directories
 - Removal of old backups
- Database Purging:**
 - Archiving and removing inactive databases based on criteria:
 - 3 months inactive with ≤ 10 records
 - 6 months inactive with ≤ 50 records
 - 1 year inactive with ≤ 200 records
 - Notifying owners of large inactive databases
- Import Table Management:**
 - Archiving and compressing import tables older than 2 months

- Limiting the number of import tables per database

4. ****System Archive Management****:

- Compressing large system archive tables
- Purging very old archive entries

Maintenance tasks can be run manually by administrators or automated via scheduled tasks.

Sources: <FileRef file-url="https://github.com/HeuristNetwork/heurist/blob/c9f37be9/admin/utilities/purgeInactiv

Scheduled Operations

Heurist includes capabilities for scheduled operations that maintain database health and perform routine tasks.

!<FileRef file-url="https://github.com/HeuristNetwork/heurist/blob/c9f37be9/Scheduled Operations" undefined fil
flowchart TD

```
subgraph "Scheduled Operations"
    dailyCron["Daily Cron Jobs"]

    dailyCron --> updateReports["Update Scheduled Reports"]
    dailyCron --> sendReminders["Send Record Reminders"]
    dailyCron --> checkURLs["Check URLs"]
    dailyCron --> notifyArchive["Notify About Archiving"]
end
```

Scheduled operations include:

1. **Report Updates:**

- Automatic regeneration of reports according to schedules
- Notification of long-running reports

2. **Reminders:**

- Sending reminders about records based on user-defined schedules
- Aggregation of reminders by frequency

3. **URL Validation:**

- Regular checking of URLs in records and fields
- Notification of invalid URLs

4. **Archive Notifications:**

- Reminding database owners to create backups
- Sending notifications about database inactivity

Scheduled operations can be configured to run at specific intervals and can be limited to selected databases.

Sources: `admin/setup/dboperations/dailyCronJobs.php` 34-137

`admin/utilities/notifyDatabaseArchive.php` 1-63

Data Management Utilities

Heurist provides a set of utility functions that support various data management operations.

Import Utilities

The import system includes utilities for:

- File upload and storage
- Data parsing and validation
- Field mapping and transformation
- Record matching and updating

Export Utilities

Export utilities support:

- Structured data output (XML, SQL, TSV)
- File packaging and compression
- Archive creation and management
- Repository upload capabilities

Database Utilities

Database utilities provide:

- Database creation and manipulation
- Structure validation and verification
- File system management
- Maintenance and cleanup operations

Sources: `hserv/utilities/DbExportTSV.php` `hserv/utilities/DbUtils.php`

`hserv/utilities/DbVerify.php`

`hserv/utilities/DbVerifyURLs.php`

`hserv/utilities/UArchive.php`

